

Inventario de datos — Urbi (Medellín Social)

Generado 2026-08-24 vía inspección directa de la DB local (`social` en `localhost:5433` , 1.5 GB) y grep de uso real en `api/` y `src/` . Conteos son `SELECT COUNT(*)` real, no `pg_stat` (que estaba desactualizado — mostraba 0 en decenas de tablas que sí tienen datos).

Leyenda de estado:

- **Vivo** — cargado, calculado y consumido por al menos un endpoint/UI.
- **Calculado pero huérfano** — pipeline completo (raw → analytics) corre y tiene datos, pero ningún endpoint ni componente lo lee.
- **Vacío / stub** — tabla existe, 0 filas o pipeline sin correr.

Universo de referencia: `raw.barrios` = **610 barrios**. Donde aplica, se da cobertura = filas / 610 — esa es la respuesta real a "qué tan completo", un conteo crudo por sí solo puede engañar (269 filas suena bien hasta que ves que son 44% del universo).

Alcance de este documento: cubre capas de datos/inteligencia urbana (catastro, tráfico, verde, seguridad, POIs, listings, scoring, comunidad). Excluye tablas transaccionales/de aplicación (`usuarios` , `agent` , `suscripciones_*` , `deals` , `favoritos` , `visita_solicitud` , `leads` , etc. — ~50 tablas de CRUD de producto, no de inteligencia geoespacial). Avisa si también quieres inventario de esas.

1. Catastro (avalúos y predios)

Tabla	Filas	Estado
<code>raw.catastro_medellin</code>	1,129,306	●
<code>analytics.listings_vs_catastro</code>	77,925	●
<code>analytics.catastro_comunas_stats</code>	20	●

Fuente: catastro Medellín, dos hojas fusionadas 2026-08-23 (`load_catastro_medellin.py` + `load_catastro_medellin_vigente.py`), discriminadas por `cd_vig_pred='S'` = vigente. Hoja1 y Hoja2 son ~99% predios distintos, no snapshots del mismo universo — no se dedupli, se preservaron ambas.

dbt agrega por comuna en `listings_vs_catastro` (avalúo m²) → `api/routers/listings.py` lo joinea a cada listing individual (`avaluo_m2_catastro` por comuna) para mostrar precio de venta vs. avalúo catastral en el listing. `validate_catastro_join.py` verifica cobertura del join `barrios.comuna` → `catastro.ds_comuna` .

Gap: el avalúo se agrega solo a nivel comuna (16 comunas), no a nivel predio/barrio — con 1.1M predios geolocalizados se podría bajar la granularidad, no se está usando.

2. Compraventas (ORIPS/SNR)

Tabla	Filas	Estado
<code>raw.compraventas_orips</code>	594,911	● huérfana en raw
<code>analytics.compraventas_municipio_stats</code> (matview)	100	● huérfana
<code>staging.stg_valorizacion</code> (dbt)	115	● (vía proyecciones)
<code>analytics.proyecciones_valorizacion</code>	6	●

Fuente: SNR Valle de Aburrá vía `load_compraventas_orips.py` — carga cruda auditable (no borra ceros ni el outlier de \$732B, no dedupe, encoding latin-1 con gotchas documentadas). Alimenta `stg_valorizacion` → `proyecciones_valorizacion`, que sí está wired (`api/routers/listings.py`, `oportunidades.py`, `barrios.py`, `comunas.py`).

Pero el matview `compraventas_municipio_stats` (100 filas, refrescado tras cada carga) y la tabla raw en sí (595k transacciones reales de compraventa) **no tienen ningún consumidor** — ni API ni dbt las leen directamente. Solo `stg_valorizacion` usa una porción para valorización, el resto del detalle transaccional está sin explotar.

3. Tráfico / congestión (HERE Flow API)

Tabla	Filas	Cobertura	Estado
<code>raw.trafico_muestras</code>	599	—	●
<code>analytics.barrios_trafico_perfil</code>	577	95% de 610 barrios	● conectada 2026-08-24

Pipeline completo y documentado (memoria `project_congestion_here.md`): modelo de **campaña estática**, no tiempo real — `trafico_muestrear.py` (muestrea `jamFactor` vía HERE Flow v7 en `pico_am/pico_pm/valle`) → `compute_trafico_perfil.py` (agrega a nivel barrio: `nivel_trafico` bajo/medio/alto, `jam_prom`, ventana pico) → se apaga el muestreo tras la campaña.

Actualizado 2026-08-24: estaba huérfana (cero consumidores) al momento del primer inventario. Se conectó el mismo día — `api/routers/barrios.py` ahora hace JOIN a `barrios_trafico_perfil` y expone `trafico` en `BarrioResponse`. Se muestra en `FloatingPanel.tsx` (sección Tráfico del panel de barrio) y en `ListingDrawer.tsx` (tile "Tráfico" dentro de "Entorno", vía el mismo endpoint `/barrios/{id}` que ya usaba `GettingAround`). No se tocó `listings.py` / `barrios_contexto` — no hizo falta duplicar el dato en un segundo JOIN.

4. Zonas verdes

Tabla	Filas	Cobertura	Estado
<code>analytics.barrios_verde</code>	610	100% de 610 barrios	●

Fuente: `raw.pois` filtrado `tipo='parque'` (OSM vía Overpass) + geometría de `raw.barrios`. Metodología simple: `n_parques` dentro del polígono × 5000 m² estimado por parque ÷ área del barrio =

`indice_verde_pct`, categorizado MUY VERDE/VERDE/MODERADO/POCO VERDE con score 15/10/6/2.

Wired en `api/routers/barrios.py` (`Verde` model, `indice_verde_pct`, `categoria_verde`, `score_verde`) → se muestra en `FloatingPanel.tsx` y participa en `barrios_score` / `pts_verde_nomada`.

Nota de calidad: el área de parque es una **estimación fija** (5000 m² por parque, no el polígono real del parque en OSM) — funcional pero burdo. Mejora obvia si se quiere precisión: usar `way` /polígonos de parque de OSM en vez de contar puntos.

5. Seguridad / criminalidad

Tabla	Filas	Cobertura	Estado
<code>raw.criminalidad</code>	25,733	— (nivel comuna, no barrio)	●
<code>analytics.barrios_seguridad</code>	269	44% de 610 barrios	●
<code>staging.stg_seguridad</code> (dbt)	21	—	●

Fuente: medata.gov.co (datasets `wtnd-h3vi` anual, `gcyu-chif` mensual), por comuna — fusionadas 2026-08-23 en una sola tabla `raw.criminalidad` (antes 3 tablas separadas).

`load_seguridad_otros_municipios.py` cubre municipios fuera de Medellín.

Por qué 44% y no 100%: la fuente es por comuna (subdivisión oficial de Medellín) y se propaga a los barrios dentro de cada comuna — barrios de municipios sin dataset de criminalidad publicado (fuera de Medellín, salvo lo que cubra `load_seguridad_otros_municipios.py`) simplemente no tienen fila. Es un gap real de fuente, no un pipeline a medio correr.

El más usado de todos: aparece en `cache.py`, `asignador_service.py`, `personalizacion.py`, routers `comparador`, `realtor`, `listings`, `barrios`, `calculadora`, `admin` — y en frontend en `ListingDrawer`, `MapFilterBar`, `FloatingPanel`, dashboard `realtor`, `comparador`, `publicar`. Alimenta `scoring`, filtros de mapa, y la pestaña Inteligencia del dashboard de `realtor` (que reusa el motor de `barrios.py`, no duplica queries — ver memoria `project_realtor_inteligencia_completa.md`).

Nota heredada de memoria: había un TODO pendiente en la sección de seguridad de Inteligencia — no verificado en esta pasada, revisar si sigue abierto.

6. POIs (metro, parques, malls, universidades, hospitales, etc.)

Tabla	Filas	Cobertura	Estado
<code>raw.pois</code>	3,334	— (Valle de Aburrá, 10 municipios)	●
<code>analytics.barrios_pois_distancia</code>	610	100% de 610 barrios	●

Fuente: OpenStreetMap vía Overpass API, bbox Valle de Aburrá completo (10 municipios). Dos loaders:

`load_pois.py` (5 tipos originales: metro, parque, mall, universidad, hospital) y

`load_pois_overpass.py` (extendido: + cafe, coworking, gimnasio, restaurante, bar, yoga_studio), upsert por `osm_id` en la misma tabla.

Wired en `barrios.py` (`LEFT JOIN analytics.barrios_pois_distancia`) → `FloatingPanel.tsx` , `MapView.tsx` , `LandingMapHeader.tsx` , y las páginas de negocio local (`local-business/$barrio_slug.tsx`).

7. Estratos y POT (uso de suelo)

Tabla	Filas	Estado
<code>raw.estratos</code>	21	
<code>raw.estratos_manzana</code>	32,384	 solo dbt
<code>raw.pot_usos_medellin</code>	25,446	 huérfana
<code>raw.indices_precio_vivienda</code> (DANE)	612	

`indices_precio_vivienda` (índice de valorización DANE por año/fuente) alimenta el LATERAL JOIN de apreciación en `api/routers/calculadora.py` (última cifra disponible por `fuente='dane'`). Wired, uso puntual.

`estratos` (nivel comuna/barrio) está ampliamente wired: `cache.py` , `models/listing.py` , routers `comparador/barrios/listings/calculadora/listings_propios` , y en casi todo el frontend (`MapFilterBar` , `ListingDrawer` , `MLSPanel` , `comparador` , `mapa` , `publicar` , `planes`).

`estratos_manzana` (nivel manzana, más fino, reproyectado EPSG:9377 → WGS84) solo lo lee `dbt/models/analytics/score_largo_plazo.sql` — wired indirectamente pero infrautilizado dado el nivel de detalle que ofrece.

`pot_usos_medellin` (uso de suelo del Plan de Ordenamiento Territorial, mismo loader que `estratos_manzana`) **no tiene ningún consumidor** — ni dbt ni API. Cargado y nunca leído.

8. Listings (todas las fuentes, vía `stg_listings_unificado`)

Tabla fuente	Filas	Estado
<code>raw.listings_metrocuadrado</code>	91,033	● unido
<code>raw.listings_fincaraiz</code>	60,490	● unido
<code>raw.listings_habi</code>	120	● unido (recién conectado, commit 782ef1b)
<code>raw.listings_casadolcecasa</code>	290	● unido
<code>raw.listings_premium</code> (VRBO)	36	● unido
<code>raw.listings_renta_media</code>	1,717	● unido
<code>public.listings_propios</code>	0	● legacy, reemplazado por <code>public.listing</code>
<code>public.listing</code>	17	● seed demo (pipeline Propio Pro real, poco volumen aún)
<code>staging.stg_listings_unificado</code>	98,388	● tabla central
<code>analytics.listings_georef</code>	82,884	●
<code>raw.listings_precio_historial</code>	144,819	⚠ cargada, no confirmado su uso en UI (historial de precio en drawer — pendiente verificar)
<code>raw.listing_media_mirror</code> (R2 webp)	0	● pipeline construido, backfill pendiente (gap conocido, ver memoria)

Las 6 fuentes activas se unen en `cache.py` vía `UNION ALL` hacia `stg_listings_unificado`, que alimenta prácticamente todo: mapa, comparador, drawer, scoring.

9. Airbnb / renta corta y media

Tabla	Filas	Cobertura	Estado
<code>raw.airbnb_listings_portal</code>	623	—	●
<code>raw.airbnb_calendar_portal</code>	7,175	—	●
<code>raw.airbnb_barrios</code>	284	47% de 610	●
<code>raw.airbnb_mercado_valle</code>	32	— (municipios, no barrios)	●
<code>analytics.barrios_airbnb_real</code>	59	10% de 610	●
<code>analytics.barrios_renta_media</code>	207	34% de 610	●
<code>analytics.barrios_renta_segmentada</code>	85	14% de 610	●
<code>analytics.barrios_amenities</code>	142	23% de 610	●
<code>analytics.barrios_mercado</code>	593	97% de 610	●

Confirmado corregido (memoria `project_airbnb_shortterm_gap.md`): renta media/premium sí está unida al feed de listings. Todo este bloque wired en `cache.py` / `barrios.py` / `calculadora.py`, expuesto vía Inteligencia del dashboard realtor.

Por qué las coberturas bajan tanto (10–34%) en vez de acercarse a 100%: a diferencia de seguridad/verde/POIs (que son datos oficiales/geoespaciales disponibles para todo barrio), Airbnb real (`barrios_airbnb_real`) y renta media/segmentada solo tienen fila donde **existe actividad real** de ese tipo de renta — un barrio sin oferta Airbnb activa o sin listings de renta media no genera agregado. No es un pipeline a medio correr, es que la mayoría de los 610 barrios no tienen ese mercado. `barrios_mercado` (97%) consolida lo que sí hay de mercado general, por eso es la cifra alta del grupo.

10. Scoring / oportunidades (capa de síntesis)

Tabla	Filas	Estado
<code>analytics.barrios_score</code>	593	●
<code>analytics.barrios_score_consolidado</code>	593	●
<code>analytics.score_corto_plazo</code>	593	●
<code>analytics.score_mediano_plazo</code>	593	●
<code>analytics.score_largo_plazo</code>	593	●
<code>analytics.barrios_oportunidades</code>	593	●
<code>analytics.barrios_liquidez</code>	581	●
<code>analytics.barrios_top10_yield</code>	10	●
<code>analytics.barrios_medianas</code>	2,977	●
<code>analytics.barrios_contexto</code>	597	●
<code>analytics.barrios_cd</code>	271	●

Capa mejor cubierta: agrega todo lo anterior (excepto tráfico y POT, que no entran al score) en scores de horizonte corto/mediano/largo plazo por barrio. Consumida en `oportunidades.py` , `favoritos.py` , `comunas.py` , `comparador` , `calculadora` , dashboard realtor.

11. Comunidad (eventos, negocios locales, noticias)

Tabla	Filas	Estado
<code>public.tiendas</code>	12,572	●
<code>public.eventos</code>	1,262	●
<code>public.noticias</code>	22	●

Fuera del scope geoespacial/urbano pero completamente wired: `admin.py` , `business.py` , `comunidad.py` , `useTiendas.ts` , `OnboardingModal.tsx` , páginas de negocio local. Ver memoria `project_comunidad.md` / `project_eventos_scrapers.md`.

Resumen — qué está realmente huérfano (calculado y sin usar)

1. `analytics.barrios_trafico_perfil` — ya no aplica, conectado 2026-08-24 (ver §3).
2. `raw.pot_usos_medellin` (25,446 registros de uso de suelo POT) — cargado, nunca leído.
3. `raw.compraventas_orips` (594,911 transacciones reales) — solo una fracción alimenta valorización vía dbt; el detalle transaccional crudo no se explota.
4. `analytics.compraventas_municipio_stats` (matview, 100 filas) — refrescada tras cada carga, sin consumidor.
5. `raw.estratos_manzana` (32,384, nivel manzana) — solo un modelo dbt lo toca; podría enriquecer más vistas dado el nivel de detalle.
6. `raw.listing_media_mirror` — tabla vacía, pipeline (mirror a R2 en webp) construido pero sin backfill (gap ya conocido).
7. `public.listings_propios` — 0 filas, legacy muerto, reemplazado por `public.listing`.

Lo mejor cubierto

Seguridad, estratos, POIs, zonas verdes, listings unificados y la capa de scoring están todos conectados de punta a punta: raw → dbt/analytics → API → frontend. Catastro está conectado pero infrautilizado (agregado a nivel comuna cuando hay detalle a nivel predio). Airbnb/renta media está completo tras la corrección de agosto.